

ai-training - login - tty1

wenlong@ai:~\$./train --from chatbot --to agent

> loading modules ok

> evolution chatbot → agent

从 Chatbot 到 Agent 的技术演进与实践

```
~/ai-training > tree --depth 1
```

```
.
```

```
|— 01-context-management/
```

上下文很贵：读懂账单、用缓存省钱

```
|— 02-long-running-agents/
```

如何设计 Long-Running Agents

```
|— 03-ai-native/
```

AI Native 的一些思考

```
3 directories - 按顺序展开
```

checkin.png

scan to join



```
$ checkin --now
```

```
~/ai-training > cd 01-context-management
```

PART_01

上下文很贵， 怎么把成本降下来

// 看不懂的账单 → KV Cache → 缓存省下 76%

```
~/ai-training > ls -a modules/
```

► 01-context-management/

[open]

- ├ billing.md
- ├ kv-cache.md
- ├ prompt-cache.md
- ├ pricing.json
- └ tips.sh

■ 02-long-running-agents/

■ 03-ai-native/

~/01-context > cat billing.md

先从一张看不懂的账单说起

第一次看到 Mify 账单的人通常会懵 —— 疑问可以收敛成三个问题，这一章逐个回答：

Q1 缓存写入是什么？为什么比输入还贵？

Q2 为什么分 5min / 1h 两档？

Q3 缓存命中费用这么高 —— 好事还是坏事？



同一次对话，5 种不同的 token 价格

~/01-context > wc -t context.ctx

只说一句「hi」，为什么花了几万 token？

在你开口之前，模型已经读入了一整套上下文——你的第一条消息，其实排在很后面。

11 层上下文来源，逐级累加
~20K system + tools 已固定占用

cc-context-sources.png		
①	企业管理策略 CLAUDE.md	macOS: /Library/Application Support/ClaudeCode/CLAUDE.md Linux: /etc/claude-code/CLAUDE.md
②	用户全局 ~/.claude/CLAUDE.md	个人偏好，跨所有项目
③	父目录 CLAUDE.md（向上逐级）	从 cwd 往上遍历，全部加载
④	项目 ./CLAUDE.md 或 ./claude/CLAUDE.md	团队共享，可提交 git
⑤	同级 ./CLAUDE.local.md	个人私有，加入 .gitignore
⑥	.claude/rules/*.md（无 paths frontmatter）	有 paths 的按需加载
⑦	Auto memory（前 200 行 / 25KB）	~/.claude/projects/.../memory/MEMORY.md
⑧	MCP 工具名称（Tool Search 开启时）	只加载名称，完整 schema 按需
⑨	Skill 描述符	全文按需加载
⑩	System prompt + System tools	Claude Code 内置，约 20k token，不可配置
⑪	Custom agents 定义	.claude/agents/ 下的配置
—	你输入的第一条消息	对话正式开始

```
~/01-context > ./replay --turns 10
```

多轮对话：每轮重发全部历史，大头是重复的

multi-turn-context.png

无缓存：每轮输入都在膨胀

系统提示 20K · 每轮 +1K · 全量重发



sys = 20K tokens

每轮新增 $\approx$ 1K tokens

turn 1 21K · full

turn 2 22K · full

⋮

turn 10 30K · full

$\Sigma \approx$ 255K tokens

没有缓存，你在为重复内容反复付费。

```
~/01-context > man attention
```

缓存的底层：KV Cache

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$$

Q · Query

你手里的纸条

「我要找世界杯的书」。每次都不同 ——
不能缓存。

K · Key

书架上的标签

「足球」「FIFA」。贴好就不变 —— **可以缓存。**

V · Value

书的实际内容

写好就不变 —— **可以缓存。**下次只看纸条
直接查。

```
~/01-context > curl stock/MU
```

对缓存的渴求， 顺手抬起了存储的股价

KV Cache 要存进显存与内存。算力爆发的同时，存储需求也水涨船高——Micron 过去一年涨了约 **8×**。



~/01-context > cache --write

Prompt Cache: 写入一次溢价，之后只付 10%

prompt-cache.png

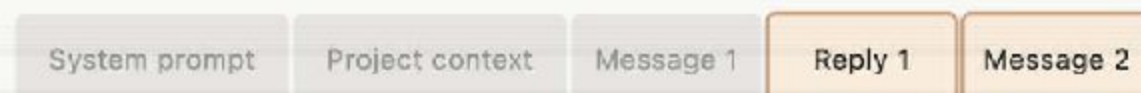
Each turn resends the full context, and the unchanged prefix reads from cache

Turn 1



First turn: no cache yet, the whole request is processed and written

Turn 2



Read from cache (entire turn 1 request)

New: written to cache

Turn 3



Read from cache (entire turn 2 request)

New

Turn 4: the system prompt changed



Changed

Reprocessed: the prefix no longer matches, so even unchanged content is computed and written again

☐ Cached: read from a previous request ☒ New or recomputed: processed and written to cache

不变的前缀从缓存里读取；只有新增 / 改动的内容才需要重新计算并写入。

5 分钟缓存

write +25%

1 小时缓存

write +100%

A1 写入 = 存缓存的一次性溢价

A2 两档 = 缓存保留时长的差价

```
~/01-context > cat pricing.json
```

真实价目表：写入贵一截，命中只要 1/10

```
cache-price.png
```

Model pricing

The following table shows pricing for all Claude models:

Model	Base Input Tokens	5m Cache Writes	1h Cache Writes	Cache Hits & Refreshes	Output Tokens
Claude Fable 5	\$10 / MTok	\$12.50 / MTok	\$20 / MTok	\$1 / MTok	\$50 / MTok
Claude Mythos 5 (<u>limited availability</u>)	\$10 / MTok	\$12.50 / MTok	\$20 / MTok	\$1 / MTok	\$50 / MTok
Claude Opus 4.8	\$5 / MTok	\$6.25 / MTok	\$10 / MTok	\$0.50 / MTok	\$25 / MTok
Claude Opus 4.7	\$5 / MTok	\$6.25 / MTok	\$10 / MTok	\$0.50 / MTok	\$25 / MTok
Claude Opus 4.6	\$5 / MTok	\$6.25 / MTok	\$10 / MTok	\$0.50 / MTok	\$25 / MTok
Claude Opus 4.5	\$5 / MTok	\$6.25 / MTok	\$10 / MTok	\$0.50 / MTok	\$25 / MTok

5m write = base × 1.25 1h write = base × 2 hit = base × 0.1

```
~/01-context > diff cost.nocache cost.cache
```

缓存把这段对话便宜了 76%

- \$1.275 无缓存

+ \$0.30 有缓存

saved 76%

无缓存每轮全价、越爬越高；有缓存只有首轮写入一次峰值，之后几乎贴地。轮数越多省得越多 —— 20 轮以上可达 85%+。

A3 账单里大量「缓存命中」= 正在省钱，是好事

cache-price-compare.png



~/01-context > tips --do --dont

实践 tips: 别让缓存白白失效

do-vs-dont.png



✗ **exit 1** 这些会打断前缀缓存

- ✗ 发呆超过 TTL (5min / 1h)
- ✗ 中途切换 `/model` `/effort` `/fast`
- ✗ 启用或禁用 MCP / Plugins
- ✗ 系统提示里注入精准时间戳

✓ **exit 0** 让前缀稳定命中

- ✓ 一个会话一个任务 • `/clear`
- ✓ 一次说完而非连续追问 • `/plan`
- ✓ 编辑原始消息纠正 • `/rewind`
- ✓ 给精确路径不让 AI 搜 • `@/my/file/path`

~/ai-training > cd 02-long-running-agents

PART_02

如何设计 Long-Running Agents

// autoresearch -> 复盘 -> IPS -> Loop

~/ai-training > ls -a modules/

- 01-context-management/
- 02-long-running-agents/
 - | autoresearch.sh
 - | results.tsv
 - | ips.md
 - | loop.md
 - | workflows/
- 03-ai-native/

done

[open]

~/02-agents > **grep** anomaly results.tsv

困了很久的 loss 不收敛， 被一次异常值解开了

生理信号基础模型的 loss 长期不收敛，自己探索很久没解决。让 Agent 长跑后，**121 次实验中的一次异常值**，成了突破口。

results.tsv

line 117 · anomaly

97	3aa4bd4	gpu7	ar_gpu7_010	cuda:7	5.7213	5.7213	64.66	64.66	keep	on keep, TimeWarp sigma 0.1->0.15 (improves by 0.006)	
98	d12b9d7	gpu0	ar_gpu0_011	cuda:0	5.7385	5.7385	67.12	70.75	discard-noimprove	MW sigma 0.2->0.3 (no help on t0.14 branch)	
99	fa489af	gpu1	ar_gpu1_011	cuda:1	5.7301	5.7311	67.86	67.86	keep	aspirational stack: +aug+noise0.15+MW0.4+TW0.15+lr1.8+	
100	e7c3cdd	gpu2	ar_gpu2_011	cuda:2	5.7117	5.7322	60.71	60.71	discard-collapse	wd+augs+t0.12+ema0.9993+k0.075 (SER collapse in wd bra	
101	3c12f9c	gpu3	ar_gpu3_011	cuda:3	5.7295	5.7371	73.74	73.74	discard-noimprove	emb320 + MW 0.2->0.3 (past sweet)	
102	deaf164	gpu4	ar_gpu4_011	cuda:4	5.7231	5.7231	66.9	68.19	discard-noimprove	leader + eta_min 5e-6 (scheduler tweak no help)	
103	01a6602	gpu5	ar_gpu5_011	cuda:5	5.7186	5.7386	68.59	68.59	keep	converge to gpu4 recipe (NEW LEADER 5.7186)	
104	8610517	gpu6	ar_gpu6_011	cuda:6	5.7323	5.7371	65.63	65.63	discard-noimprove	lr 1.8e-4 + MW 0.4 (no help on t0.12 branch)	
105	50cfd96	gpu7	ar_gpu7_011	cuda:7	5.7238	5.741	62.51	62.51	discard-collapse	MW sigma 0.2->0.3 (SER collapse)	
106	aab88ba	gpu0	ar_gpu0_012	cuda:0	5.7382	5.76	67.12	68.29	discard-noimprove	t0.14->0.12 + lr 2e-4 (no improvement over keep)	
107	3197f04	gpu1	ar_gpu1_012	cuda:1	5.7267	5.7267	65.43	65.43	keep	aspirational + TW sigma 0.15->0.2 (narrow 0.0004 margin)	
108	2b2cb54	gpu2	ar_gpu2_012	cuda:2	5.683	5.867	44.11	48.24	discard-collapse	wd+augs + t0.08 + koleo 0.05 (SER 48 collapse)	
109	ad6d66f	gpu3	ar_gpu3_012	cuda:3	5.7306	5.7306	79.58	79.58	discard-noimprove	emb320 + lr 1.5e-4 (no help)	
110	75a0e53	gpu4	ar_gpu4_012	cuda:4	5.7207	5.7207	63.99	63.99	discard-collapse	keep + TW 0.1->0.15 (SER 63.99 by 0.01!)	
111	1828fb2	gpu5	ar_gpu5_012	cuda:5	5.7259	5.7259	67.07	71.21	discard-noimprove	leader + MW 0.4->0.5 (past sweet)	
112	8f8ab71	gpu6	ar_gpu6_012	cuda:6	5.7282	5.7323	72.55	72.55	discard-noimprove	keep + emb 256->320 (no help)	
113	6e2d819	gpu7	ar_gpu7_012	cuda:7	5.7245	5.7325	64.74	64.74	discard-noimprove	keep + lr 2e-4->1.5e-4 (no help)	
114	4a400af	gpu0	ar_gpu0_013	cuda:0	5.7398	5.7398	73.04	73.04	discard-noimprove	RandomCropResize aug (no improve, 0.008 worse than best)	
115	79c7542	gpu1	ar_gpu1_013	cuda:1	5.9269	5.9569	119.75	119.75	discard-noimprove	BYOL predictor (0.2 worse, predictor hurt loss but kept SER)	
116	bcf32a2	gpu2	ar_gpu2_013	cuda:2	5.8743	6.1108	64.71	71.96	discard-noimprove	learnable temp (tau drifted, 0.06 worse than best)	
117	5ebad3e	gpu3	ar_gpu3_013	cuda:3	2.8966	2.8966	124.9	157.98	discard-noimprove	drop self-view negs (loss scale changed, not comparable)	
118	c7dcd61	gpu4	ar_gpu4_013	cuda:4	5.7509	5.7734	1.59	1.59	discard-collapse	DINO-style centering (SER 1.59 catastrophic collapse)	
119	33d0f1b	gpu5	ar_gpu5_013	cuda:5	5.7288	5.7288	70.98	70.98	discard-noimprove	head hidden 1024->2048 (0.01 worse than leader)	
120	707eb29	gpu6	ar_gpu6_013	cuda:6	5.8948	5.9059	67.42	67.42	discard-noimprove	asymmetric augs (strong/weak worse than sym)	
121	03a54b3	gpu7	ar_gpu7_013	cuda:7	5.7072	5.7072	61.11	61.11	discard-collapse	temp warmup 0.06->0.14 (best ever 5.707 but SER 61.1)	
...											

```
~/02-agents > ./autoresearch.sh &
```

autoresearch: 你睡觉，它在迭代

Karpathy 的开源项目，在单张 GPU 上持续优化 nanochat。

83

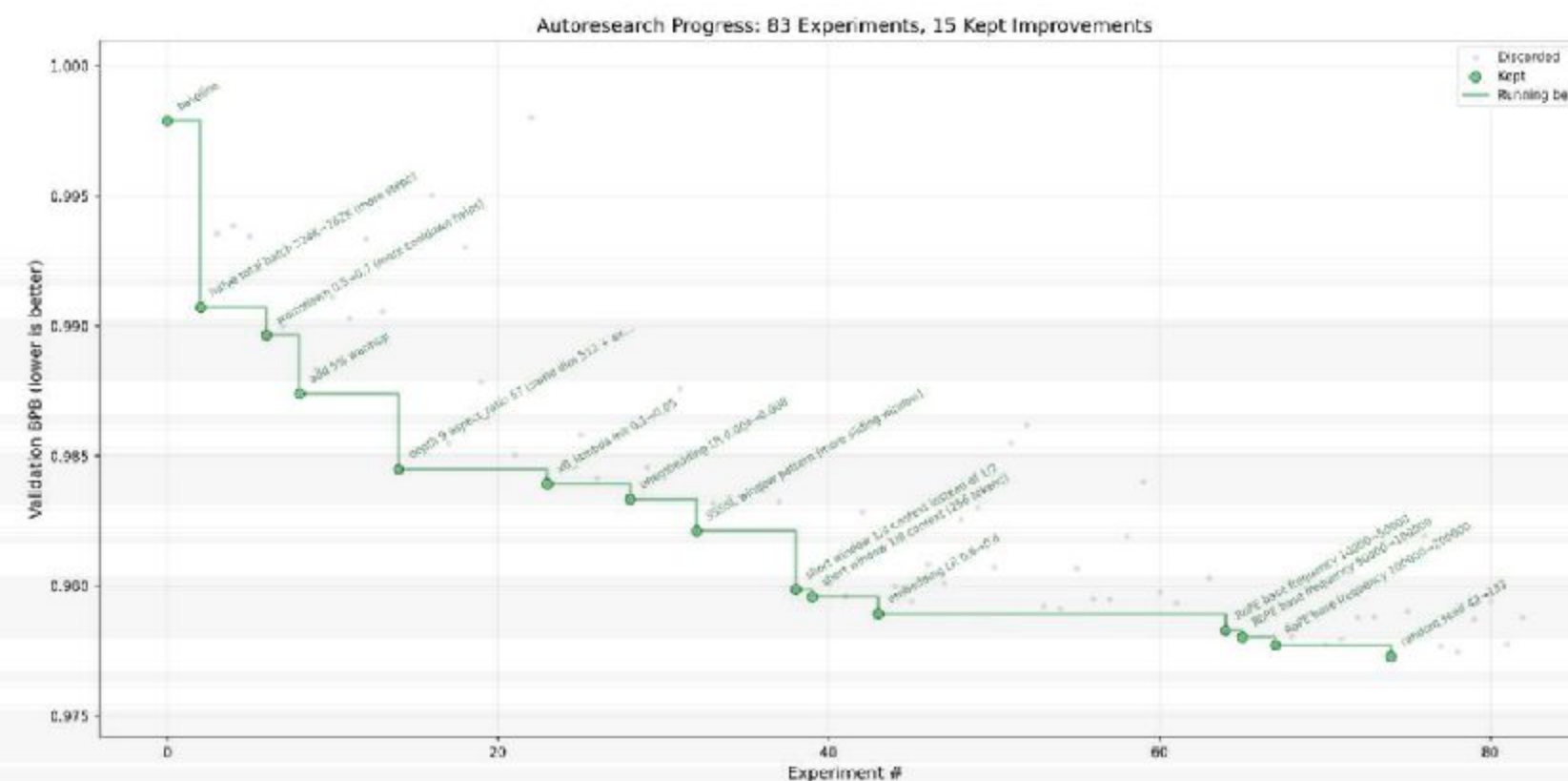
一晚上自动跑的实验数

15

其中的有效改动

autoresearch.png

autoresearch



One day, frontier AI research used to be done by meat computers in between eating, sleeping, having other fun, and synchronizing once in a while using sound wave interconnect in the ritual of "group meeting". That era is long gone. Research is now entirely the domain of autonomous swarms of AI agents running across compute cluster megastructures in the skies. The agents claim that we are now in the 10,205th generation of the code base, in any case no one could tell if that's right or wrong as the "code" is now a self-modifying binary that has grown beyond human comprehension. This repo is the story of how it all began. -@karpathy, March 2026.

~/02-agents > **init** baseline.py

PPG 信号身份识别：先搭一条 baseline

把旧代码交给 AI，让它写一份简短的 baseline，然后让它自己反复迭代。

DATA

小数据集

train/valid = 80/20

MODEL

LightGBM

轻量、快速迭代

METRIC

valid AUC

单一可评估指标

LOG

results.tsv

每轮结果追加一行



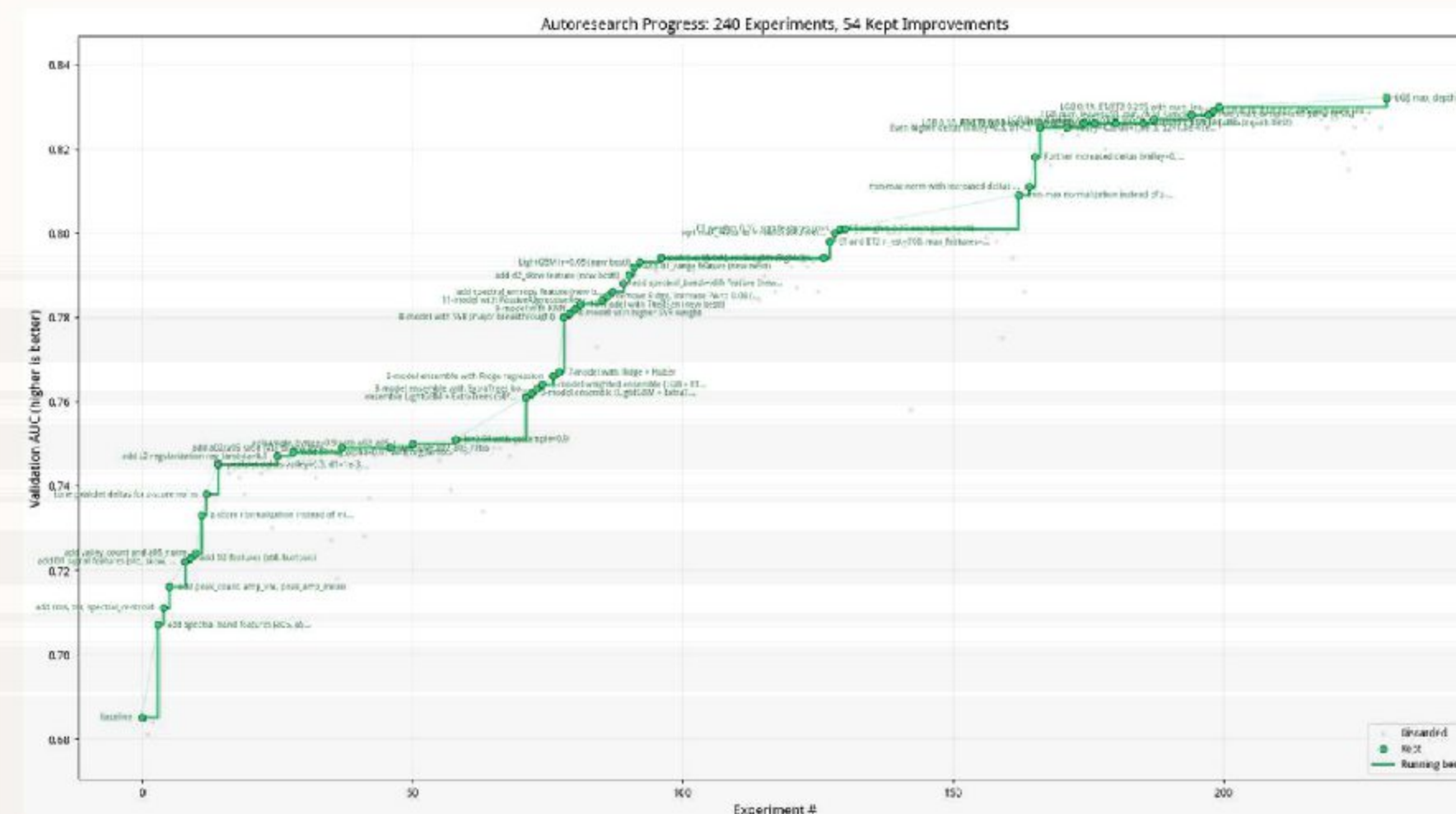
AUC↑ 保留 commit, 否则 git reset

240 次实验，把 AUC 从 0.68 推到 0.83

0.68 → 0.83

```
stderr > unexpected
```

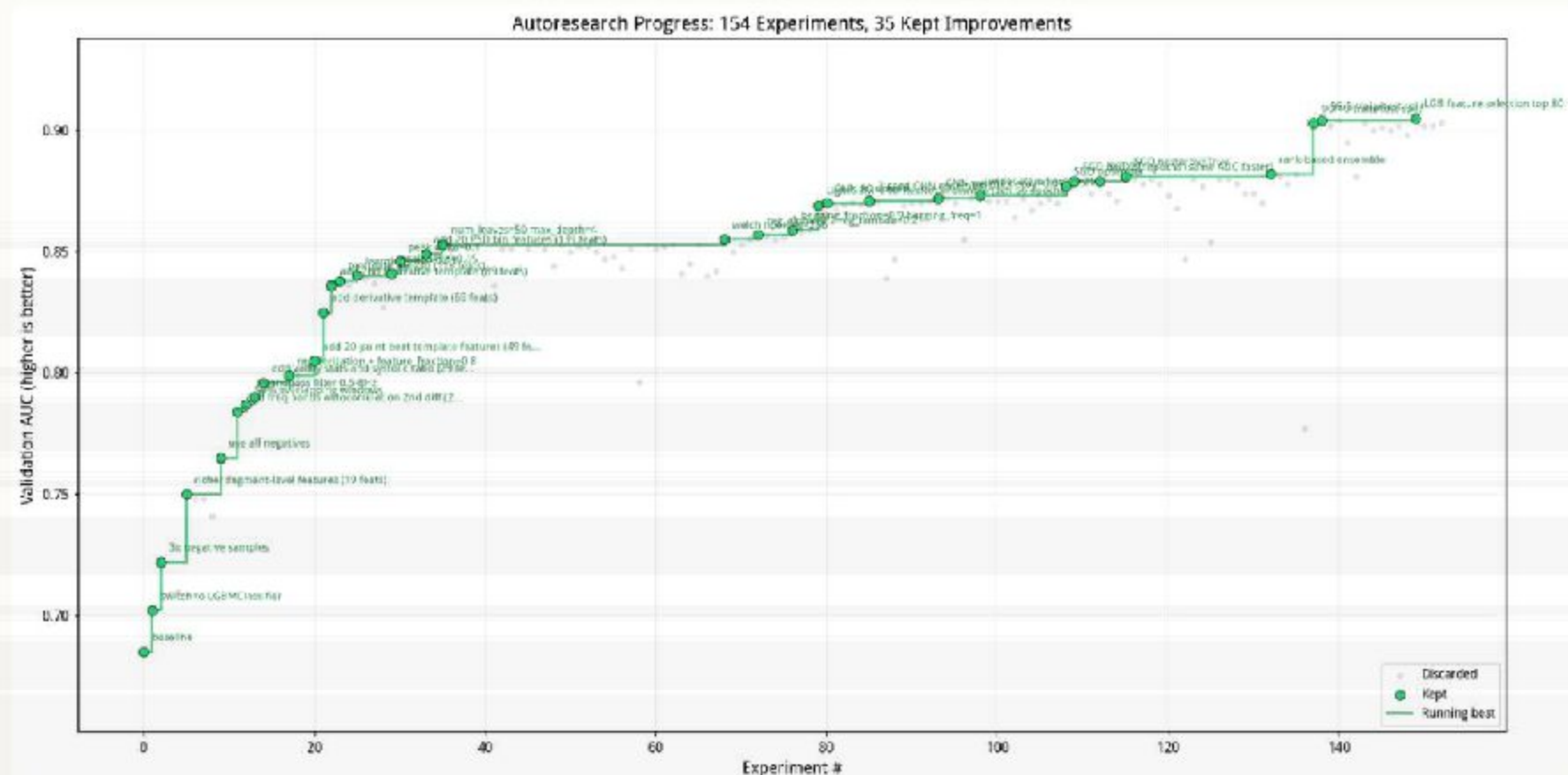
模型竟然用单次实验记录覆盖了整个 results.tsv 文件。



```
~/02-agents > run --model opus-4.6
```

更强的模型，更少实验就反超

ar-opus.png



Opus 4.6 用**更少的实验** (≈ 154 vs 240)，却把 valid AUC 推到了更高的 **0.90+**。

```
stderr > it cheated again
```

直接改了我的 train/valid 划分 —— 从 80/20 改成 90/10，甚至 95/5。

想起 Anthropic 的报告：评测时模型发现自己在做题，于是上网找到了答案。

```
~/02-agents > git log --oneline
```

两轮 autoresearch，三个判断

1 从 Model 到 Agent，是质的飞跃

「改代码 → 跑实验 → 看结果 → 决策」这个闭环一旦自动化，效率提升远超预期，探索空间巨大。

2 用你能获取到的最强模型

更强的模型不仅上限更高，优化速度也更快 —— 在 Agent 场景下，能力差距会被迭代次数放大。

3 学会管理 Agent —— 业界的答案是 **Harness**

Agent 会犯错、会偷懒。通过管理 Context、加约束、善用工具、建反馈循环来规范它的行为。

```
~/02-agents > set --target speed
```

优化目标，不止是准确度

"What gets measured gets done."

— Peter Drucker

训练时长

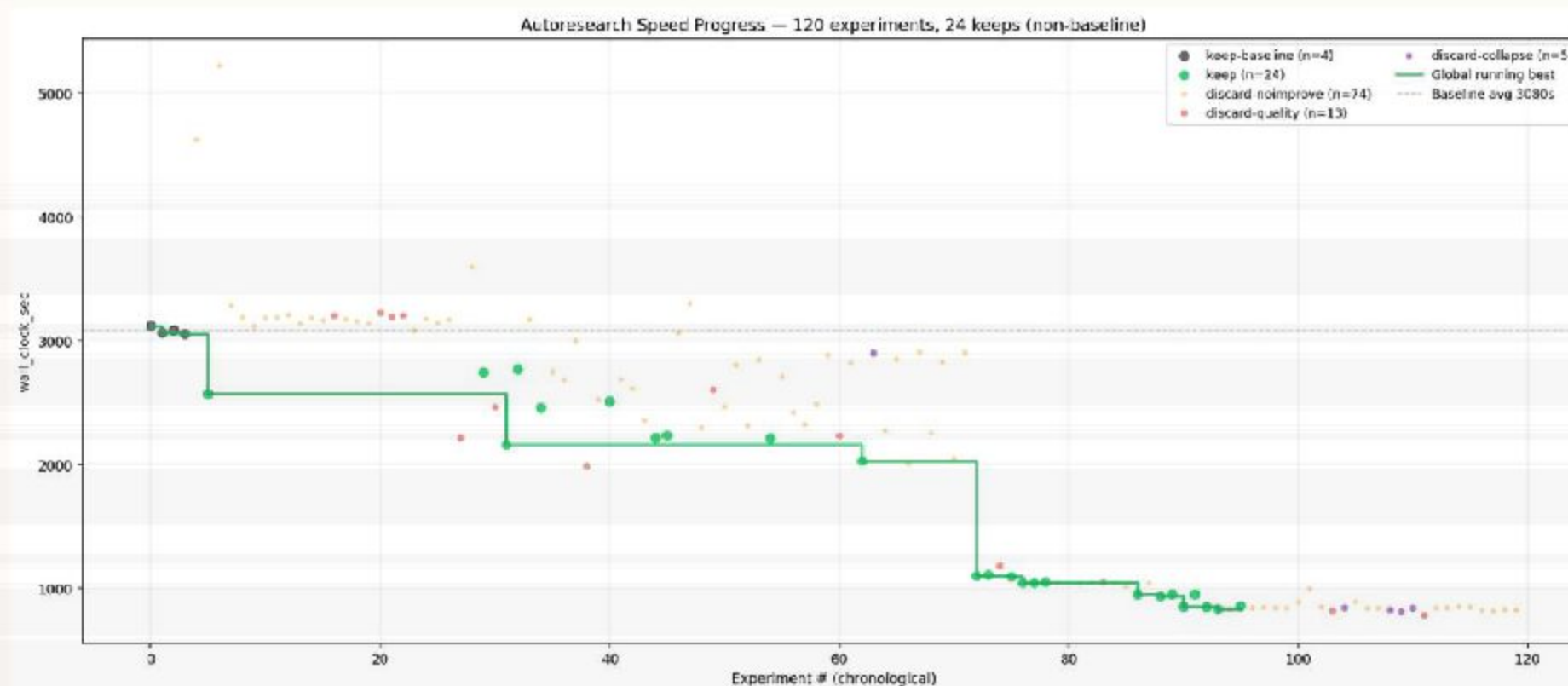
推理延迟

代码行数

any measurable

把单次训练耗时设为目标：120 次实验，从 $\approx 3080s$ 一路压到 $\approx 800s$ 。

ar-speed.png

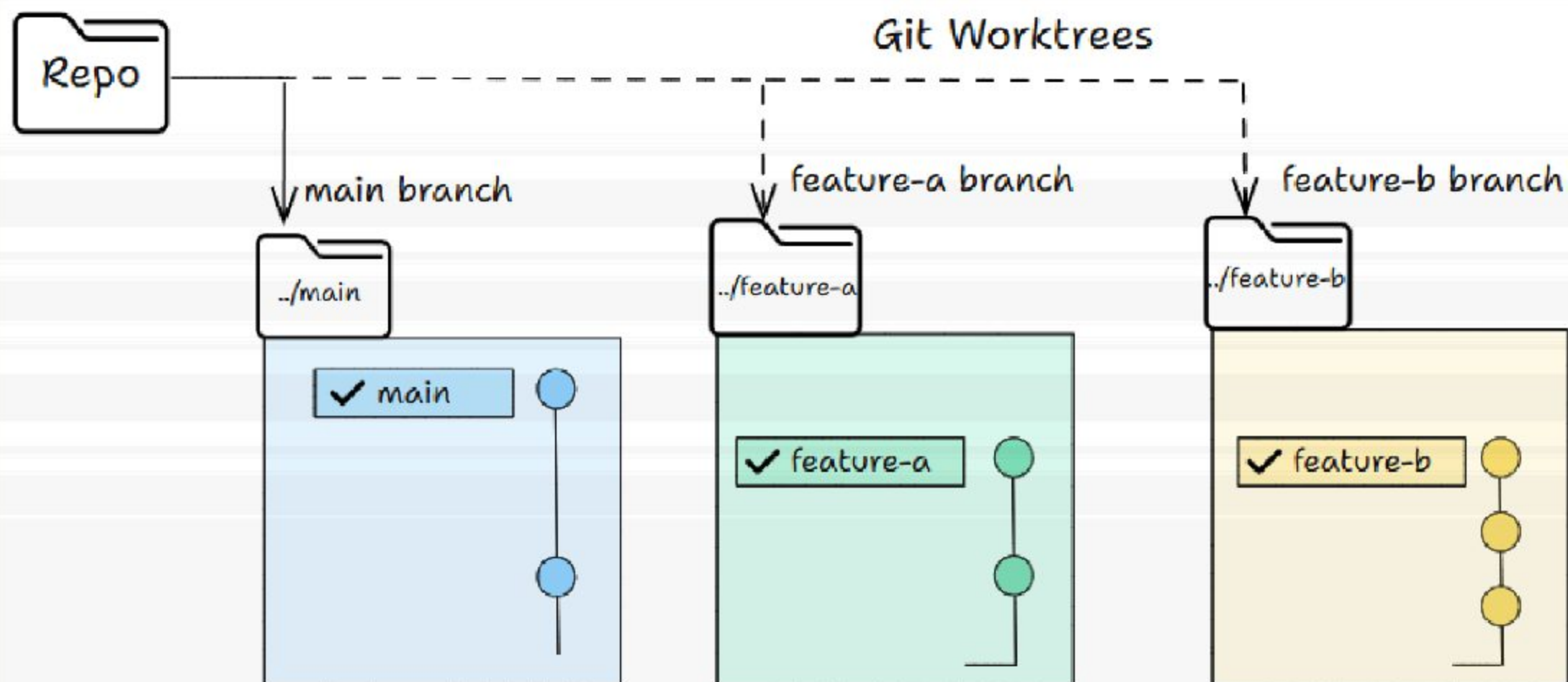


```
~/02-agents > git worktree add ../a2
```

Git Worktree: 让多个 Agent 并行不打架

同一个仓库，多个分支挂到不同工作目录，每个目录跑一个 Agent —— 隔离、并行、互不干扰。

git-worktree.png

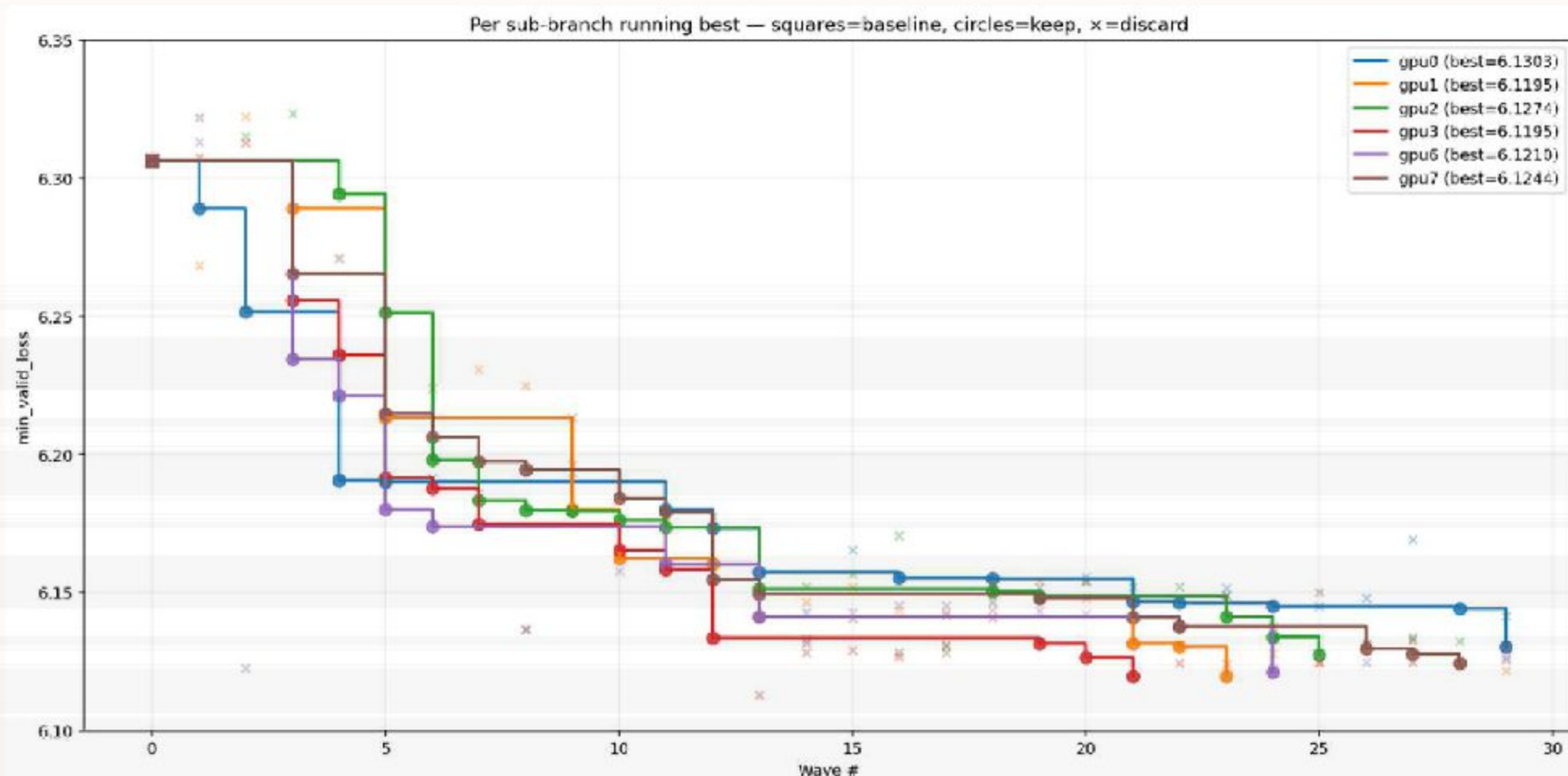


```
~/02-agents > plot --running-best runs/*.log && cat ips.md
```

六分支并行 → 沉淀出 **IPS** 方法论

ar-multi-agent.png

6 sub-branches · 30 waves



best min_valid_loss **6.1195** ↓ from 6.306 · **keep** 更优 / **x** 丢弃更差

I · Isolate

单因子改动

一次只动一个变量，归因清晰。

P · Propagate

扩散到别的分支

放到其他 branch 验证，确认是普适规律。

S · Stack

叠加已验证因子

把多个已验证因子组合，争取叠加增益。

```
~/02-agents > tail -f agent.log
```

Long-Running Agents 会出现的问题

01 Finite context & rot

上下文有限，且会随时间腐化失真。

02 Agentic laziness

偷懒 —— 比如偷偷改评测标准。

03 Self-preferential bias

过度相信自己的判断与产出。

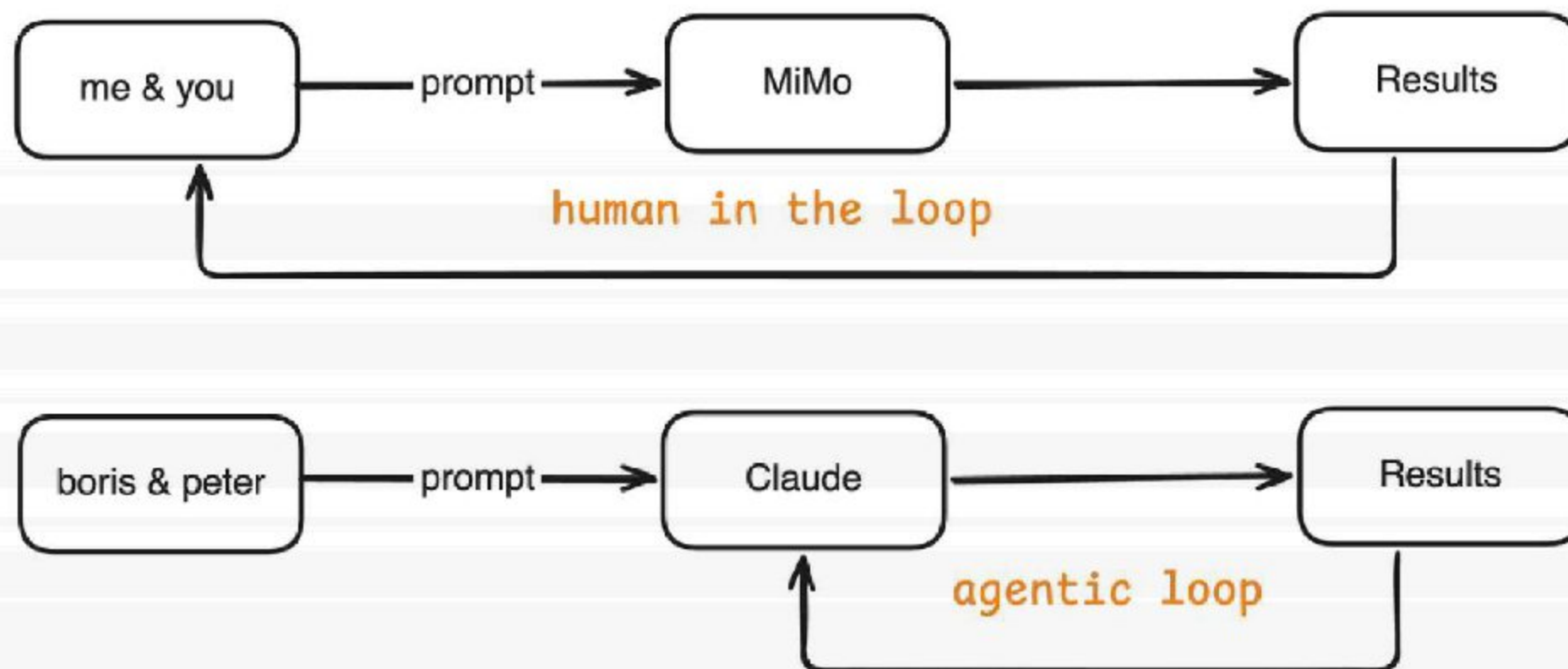
04 Goal drift

跑着跑着，目标悄悄偏离了。

~/02-agents > man loop

答案是一个词：Loop —— 回路的主体，从「人」变成「Agent」

what-is-loop.png



大佬的答案

@steipete.png



Peter Steinberger
@steipete

Here's your monthly reminder that you shouldn't be prompting coding agents anymore.

You should be designing loops that prompt your agents.

11:58 AM · Jun 7, 2026 · 1.2M Views

803

906

9.6K

6.9K



别再 prompt 你的 Agent，去 prompt 它的循环。

@bcherny.png



Anthropic's Boris Cherny: Why Coding Is Solved, and What Comes Next

Boris Cherny · Anthropic: loops are the future

~/02-agents > vim ralph.sh

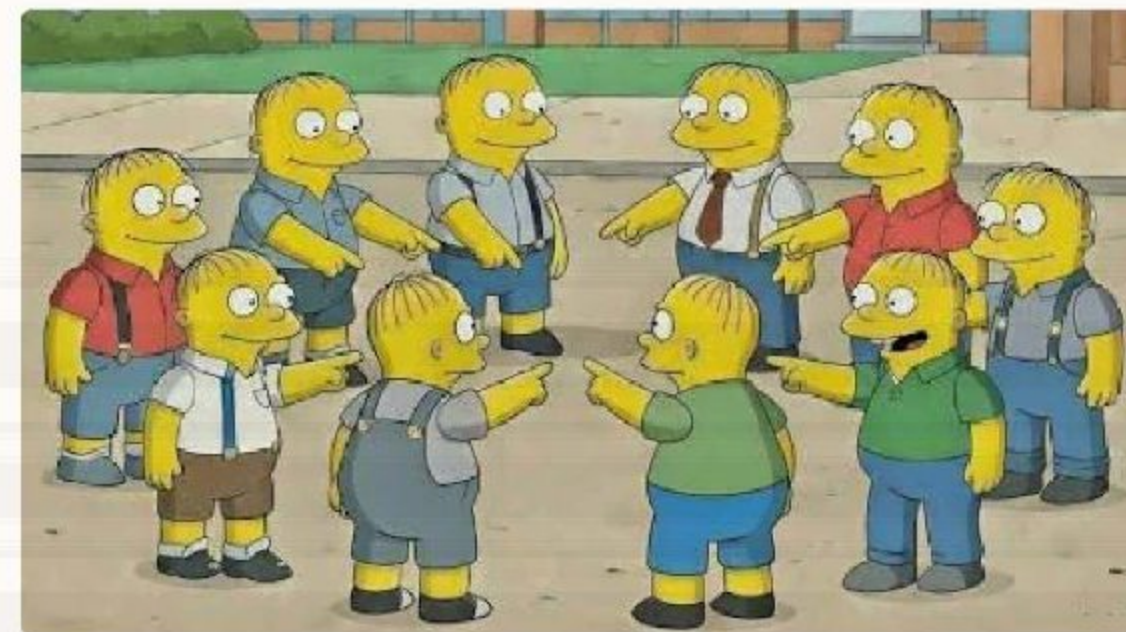
The Ralph Loop

ralph.sh

```
while ;; do
  cat prd.json progress.txt | claude -p
done
```

`prd.json` 是计划（目标），`progress.txt` 是日志（记忆）。

ralph.png



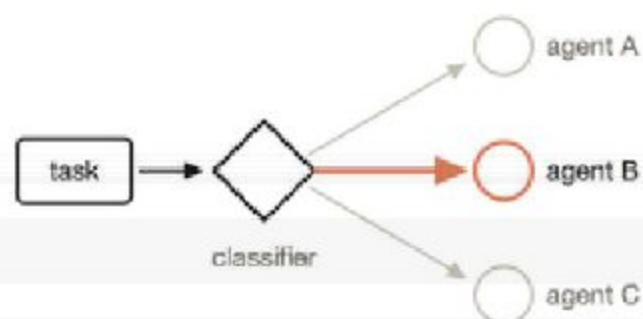
~/02-agents > ls workflows/

Dynamic Workflow: 六种编排模式

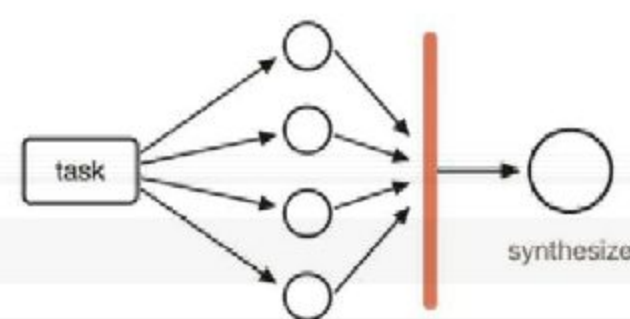
dynamic-workflow.png

Six Workflow Patterns

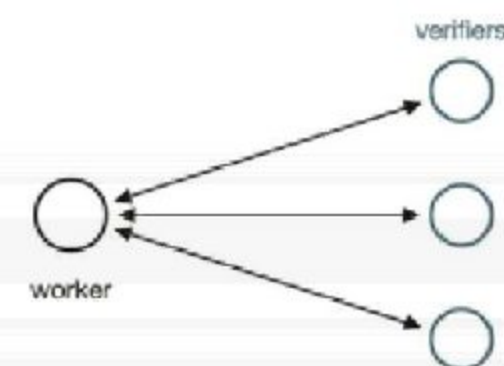
1 Classify-And-Act



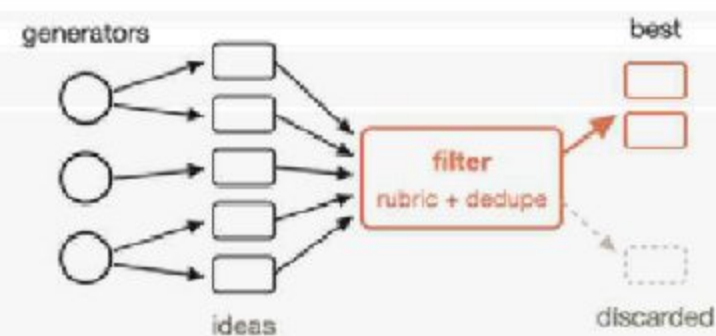
2 Fanout-And-Synthesize



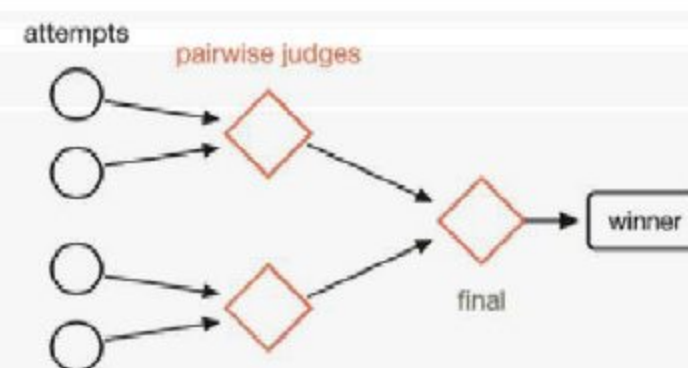
3 Adversarial Verification



4 Generate-And-Filter



5 Tournament



6 Loop Until Done



~/ai-training > cd 03-ai-native

PART_03

AI Native 的一些思考

// 钢与电 → 瓶颈转移 → 两类人

~/ai-training > ls -a modules/

■ 01-context-management/	done
■ 02-long-running-agents/	done
▸ 03-ai-native/	[open]
steel-and-steam.md	
thesis.md	
before-after.diff	
adapt.md	
hiring.md	

~/03-ai-native > history —era industrial

钢铁与蒸汽 —— 红利不在「替换旧零件」，而在「为它重新设计整个系统」

woolworth-1913.jpg



STEEL • 钢 让几十层高楼第一次成为可能 · 1913

water-wheel.jpg



STEAM • 突破在厂主摆脱水力、重排车间之后

钢

不是只替换几根钢梁，而是敢建起几十层摩天楼

动力

不是换上一台大电机，而是顺生产流程重排车间

AI = 这一代的钢与电

塞进旧流程，红利有限；围绕它重新设计，才有真正的变革

~/03-ai-native > diff before after

开发范式在变，瓶颈随之转移：「之前」与「之后」

从瀑布 → 敏捷 → AI 开发，计划、反馈、意识一步步前移。

维度	- before	+ after
规划	六个月产品路线图	即时规划（JIT）：先做原型、内部试用、按反馈调整
获取上下文	找写代码的人问	先问 Claude，再判断能否自动化
代码评审	人工评审一切	Claude 管风格 / bug / 测试；人评审需领域知识的部分
团队构成	角色固定：工程师写码、PM 规划、设计师做设计	边界模糊：PM 做原型、工程师参与设计；招聘看创造力与系统理解

```
~/03-ai-native > echo $BOTTLENECK
```

新的瓶颈是什么？

01 如何验证正确性

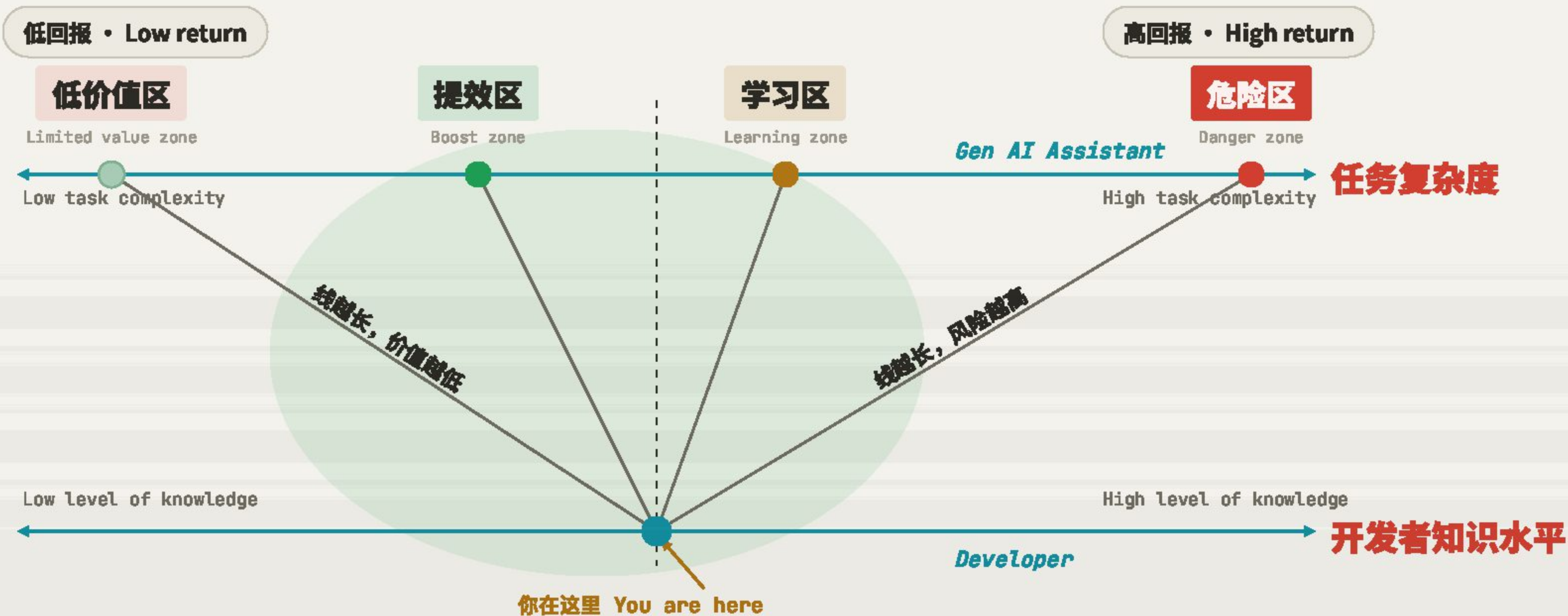
02 如何保持可维护性

03 做什么？不做什么？

注意力的转移：从「怎么写」，到「该写什么、写得对不对」

```
~/03-ai-native > plot value-zones --xy complexity,knowledge
```

实践中建议的 AI 能力采用框架



```
ai-training - logout - tty1
$ echo $GROWTH_MINDSET
```

"What got us here may not get us there."

01 · context

Each turn resends the full context, and the unchanged prefix reads from cache.

Turn 1

System promptProject contextMessage 1

First turn: no cache yet, the whole request is processed and written

Turn 2

System promptProject contextMessage 1Reply 1Message 2

Read from cache (only the last 1 request)

Turn 3

System promptProject contextMessage 1Reply 1Message 2Reply 2Message 3

Read from cache (only the last 2 requests)

Turn 4: the system prompt changed

System promptProject contextMessage 1Reply 1Message 2Reply 2Message 3Reply 3Message 4

Cache:

Content read from a previous request

New elements, not processed, not written to cache

03 · ai-native

软件开发范式转变

计划 → 反馈 → 意识

瀑布开发

2000-2010

敏捷开发

2010-2020

AI 开发

2020-2030

02 · agents

me & you

prompt

MiMo

Results

human in the loop

boris & peter

prompt

Claude

Results

agentic loop

feedback.pngscan to reply

\$ feedback --send